

Intrusion Detection Using Machine Learning

By: Zach Faulkner

Imports

My first step in creating my Random Forest and K-Nearest Neighbor Classification program was importing the necessary libraries. I used pandas to read the .csv, and sklearn to import the Random Forest and K-Nearest Neighbor classifiers, the metrics, and preprocessing tools. Because the dataset is extremely large, computing it on a normal machine is extremely resource intensive. Thus, as you will see later in the code a smaller portion of the dataset will be used to train and test the model. In order to make sure that the model is not overfitted and is able to correctly identify attacks at a high rate, I also imported DummyClassifier from sklearn to help validate the models performance by creating a minimum threshold of performance to judge the model. Lastly, I imported csr_matrix to help with the high dimensionality of the dataset and hstack to combine the numeric and encoded features.

```
MalwareClassification.py > ...
1  import pandas as pd
2  import numpy as np
3  from sklearn.ensemble import RandomForestClassifier
4  from sklearn.neighbors import KNeighborsClassifier
5  from sklearn.model_selection import train_test_split, cross_val_score
6  from sklearn.preprocessing import StandardScaler, LabelEncoder
7  from sklearn.decomposition import PCA
8  from sklearn.metrics import accuracy_score, f1_score
9  from sklearn.dummy import DummyClassifier
10 from scipy.sparse import csr_matrix, hstack
11
```

Preprocessing

The dataset I am using for this project is the [hikari-2021](#) dataset. This dataset contains a variety of benign network traffic as well as encrypted malicious traffic. The dataset contains features such as origin IP, response IP, conversation duration and much more. I have set the program to only use 10% of the dataset as it is so large that running it on normal machines is extremely resource intensive. Although this is a small fraction of the dataset, it will still give us thousands of diverse samples that can be used to train the model. Using these features I can create a model that accurately identifies malicious network traffic. As mentioned previously this dataset is extremely large, thus preprocessing this data to reduce dimensionality is paramount to the success of this model. Because this dataset contains multiple non-numeric columns, I used “x.select_dtypes(include=['object']).columns” to identify all non-numeric columns and LabelEncoder to encode categorical data. The numerical data is converted into a sparse matrix to better handle the high dimensionality of the data and then combined with the categorical data using hstack. In order to reduce dimensionality I implemented PCA to reduce the dimensionality to only 50 principal components, this vastly increases the runtime performance of the model.

The last steps of the preprocessing process was scaling the feature list to balance the feature scale and encoding the target “Label” variable. The dataset is then split into an 80/20 training-test split.

```
MalwareClassification.py > ...
13 data = pd.read_csv("hikari.csv")
14
15 #Inspect class distribution
16 data['Label'].value_counts(normalize=True).plot(kind='bar', title='Class Distribution')
17
18 #Create sample dataset
19 data = data.sample(frac=0.1, random_state=42)
20
21 #Split features and labels
22 x = data.drop(columns=["Label"])
23 y = data['Label']
24
25 #Encode non-numeric columns with LabelEncoder
26 non_numeric_cols = x.select_dtypes(include=['object']).columns
27 le = LabelEncoder()
28
29 #Use LabelEncoder for categorical data
30 for col in non_numeric_cols:
31     x[col] = le.fit_transform(x[col].astype(str))
32
33 #Convert to sparse matrix format column-wise
34 numeric_cols = x.select_dtypes(include=['int64', 'float64']).columns
35 sparse_numeric = csr_matrix(x[numeric_cols].values)
36 sparse_categorical = csr_matrix(x[non_numeric_cols].values)
37
38 #Combine numeric and categorical sparse matrices
39 x_sparse = hstack([sparse_numeric, sparse_categorical])
40
41 #Feature reduction with PCA
42 pca = PCA(n_components=50, random_state=42)
43 x_reduced = pca.fit_transform(x_sparse.toarray())
44
45 #Scale the reduced features
46 scaler = StandardScaler()
47 x_scaled = scaler.fit_transform(x_reduced)
48
49 #Encode target variable
50 y_encoded = LabelEncoder().fit_transform(y)
51
52 #Split data into training and testing sets with a 80/20 split
53 x_train, x_test, y_train, y_test = train_test_split(x_scaled, y_encoded, test_size=0.2, random_state=42)
```

Creating the Models

The next step in the project was creating the models. In order to validate the consistency of each model, I created a function named “evaluate_model”, this function will assess whether or not the models are performing across 5 different data splits (aka folds). The first four folds will be used to train the data and the fifth fold will serve as a test fold. Ideally, the results of this function should have a high accuracy and a low standard deviation. I then moved on to creating the dummy classifier threshold. This classifier will simply just calculate the accuracy percentage if the model were to guess the most frequent label every time. This will establish a minimum threshold for the model to outperform. Lastly, I created the Random Forest Classifier and K-Nearest Neighbor classifiers. I wrote a separate GridSearchCV function to test a range of hyperparameters for both classifiers in order to find the most optimal hyperparameters. In order to prioritize runtime, I deleted the GridSearchCV function from the final build, however I have left the parameter combos commented out as a reference. The classifiers are then fitted with the training data and used to predict based on the test data. The prediction data is then compared to the original test data to calculate the accuracy of the models.

```
61 #Evaluate model with cross-validation
62 def evaluate_model(model, x_train, y_train):
63     cv_scores = cross_val_score(model, x_train, y_train, cv=5, scoring='accuracy')
64     return cv_scores.mean(), cv_scores.std()
65
66 #Threshold Dummy Classifier
67 dummy = DummyClassifier(strategy='most_frequent')
68 dummy.fit(x_train, y_train)
69 dummy_acc = accuracy_score(y_test, dummy.predict(x_test))
70 print("Baseline Dummy Classifier Accuracy:", dummy_acc)
71
72 #Random Forest Classifier
73 rf = RandomForestClassifier(max_depth=20, min_samples_leaf=1, min_samples_split=2, n_estimators=50, random_state=42)
74 rf_mean_acc, rf_std_acc = evaluate_model(rf, x_train, y_train)
75 rf.fit(x_train, y_train)
76 y_pred_rf = rf.predict(x_test)
77 acc_rf = accuracy_score(y_test, y_pred_rf)
78 f1_rf = f1_score(y_test, y_pred_rf, average='weighted')
79 print("\nRandom Forest:")
80 print(f"Cross-Validation Accuracy: {rf_mean_acc:.4f} (+/- {rf_std_acc:.4f})")
81 print(f"Test Accuracy: {acc_rf:.4f}")
82 print(f"F1 Score: {f1_rf:.4f}")
83
84 #K-Nearest Neighbors Classifier
85 knn = KNeighborsClassifier(metric='manhattan', n_neighbors=5, weights='uniform')
86 knn_mean_acc, knn_std_acc = evaluate_model(knn, x_train, y_train)
87 knn.fit(x_train, y_train)
88 y_pred_knn = knn.predict(x_test)
89 acc_knn = accuracy_score(y_test, y_pred_knn)
90 f1_knn = f1_score(y_test, y_pred_knn, average='weighted')
91 print("\nK-Nearest Neighbors:")
92 print(f"Cross-Validation Accuracy: {knn_mean_acc:.4f} (+/- {knn_std_acc:.4f})")
93 print(f"Test Accuracy: {acc_knn:.4f}")
94 print(f"F1 Score: {f1_knn:.4f}")
95
```

Evaluation

This code generates the following output:

```
Random Forest:
Accuracy: 0.9945994599459946
F1 Score: 0.9944898311912473

K-Nearest Neighbors:
Accuracy: 0.9990999099909991
F1 Score: 0.9991027970693405

[Done] exited with code=0 in 28.144 seconds
```

From the metrics we can tell that the program is extremely accurate at classifying malicious traffic. The accuracy score checks if the labels predicted by the classifier algorithms match up with the true labels from the dataset, an ideal score is as close to 1 as possible. The Random Forest Classifier generated an accuracy score of 0.9946 and the K-Nearest Neighbor Classifier generated an accuracy score of 0.9991 (both rounded to the fifth digit). The second metric I used was F1 score. Because we are trying to classify malicious network traffic, we want to minimize the chance of missing a potential attack. However, we should also be checking to make sure that we are still accurate in predicting malicious activity and avoid false positives. F1 score takes both of these issues into consideration when calculating a score using the following formula, where TP = True Positives, FP = False Positives, and FN = False Negatives:

$$F1 = \frac{2 * TP}{2 * TP + FP + FN}$$

The Random Forest Classifier generated an F1 score of 0.99449 and the K-Nearest Neighbor Classifier generated a score of 0.99910. This shows that both classifiers are doing well at avoiding false positives and identifying true positives. As shown by the metrics, both models are performing extremely well at identifying malicious network traffic.